



جامعة دمشق
كلية الهندسة المعلوماتية

تقرير وظيفة البرمجة التفرعية
محرك المعالجة عالي الأداء لنظام التجارة الالكترونية

High-Performance E-Commerce Backend Engine

إعداد الطلاب

أمل محمد شعبان

عائشة محمد حسام الدالاتي

هايدي محمد خير مرعك

نور محمد الإمام

المقدمة

يهدف هذا المشروع إلى تطبيق مجموعة من المتطلبات غير الوظيفية على نظام تجارة إلكترونية مبسط مبني باستخدام Django. لا يركز المشروع على بناء متجر إلكتروني كامل بواجهة مستخدم نهائية، وإنما يركز على دراسة سلوك النظام من ناحية الأداء، التزامن، إدارة الموارد، المعالجة غير المتزامنة، المعالجة الدفعية، وتوزيع الحمل.

تم بناء النظام حول فكرة E-Commerce بسيطة تحتوي على منتجات، مخزون، طلبات شراء، وعناصر طلب. هذه الوظائف الأساسية استخدمت كأساس لتجربة المتطلبات غير الوظيفية المطلوبة. على سبيل المثال، تم استخدام عملية شراء منتج من المخزون لاختبار مشكلة Race Condition، وتم استخدام عملية دفع محاكية لاختبار إدارة الموارد، كما تم استخدام إرسال إيميل وتوليد فاتورة بشكل محاكي لاختبار المعالجة غير المتزامنة.

بعض العمليات في المشروع لم تُنفذ بخدمات حقيقية، بل تمت محاكاتها بشكل مقصود. فمثلاً، عملية الدفع لا ترتبط ببوابة دفع حقيقية مثل Stripe أو PayPal، وإرسال الإيميل وتوليد الفاتورة لا يستخدمان خدمات خارجية حقيقية. تم اعتماد هذه المحاكاة لأن الهدف الأساسي من المشروع هو اختبار المتطلبات غير الوظيفية وليس بناء نظام إنتاجي كامل أو ربطه بخدمات خارجية.

يتضمن المشروع عشرة متطلبات غير وظيفية رئيسية. المتطلبات الخمسة الأولى تعالج Race Condition، إدارة الموارد باستخدام Thread Pool، المعالجة غير المتزامنة باستخدام Queue، المعالجة الدفعية Batch Processing، وتوزيع الحمل Load Distribution. أما المتطلبات الخمسة الإضافية فتركز على Redis ACID Transaction، سلامة المعاملات Distributed Caching، Redis Distributed Lock، اختبار الضغط Stress Testing، و Benchmarking and Bottleneck Analysis.

تم اختبار كل مطلب عملياً باستخدام سكريبتات Python أو JMeter أو أوامر Django management commands، وتم توثيق النتائج باستخدام لقطات شاشة توضح الفرق بين الحالة قبل تطبيق الحل وبعده. بناءً على ذلك، يوضح التقرير ليس فقط الكود الذي تم تنفيذه، بل أيضاً سبب اختيار كل حل، وطريقة اختباره، والنتائج التي تثبت فعاليته.

نظرة عامة على المشروع

يعتمد المشروع على نظام تجارة إلكترونية مبسط تم تطويره باستخدام Django. تم اختيار هذا النوع من الأنظمة لأنه يحتوي على عمليات مناسبة لاختبار المتطلبات غير الوظيفية، مثل شراء المنتجات، خصم المخزون، إنشاء الطلبات، تنفيذ الدفع، إرسال الإشعارات، توليد الفواتير، ومعالجة تقارير المبيعات.

يتكون المشروع بشكل رئيسي من تطبيق **products** وتطبيق **orders**. يحتوي تطبيق **products** على المنتجات والمخزون، بينما يحتوي تطبيق **orders** على الطلبات، عناصر الطلب، السلة، إضافة إلى أجزاء خاصة ببعض المتطلبات مثل سجلات المعالجة الدفعية.

تم ربط كل متطلب غير وظيفي بسيناريو مناسب داخل نظام التجارة الإلكترونية. فمثلاً، استخدمنا شراء المنتج لاختبار Race Condition، واستخدمنا عملية دفع محاكية لاختبار إدارة الموارد، واستخدمنا إرسال إيميل وتوليد فاتورة لاختبار المعالجة غير المتزامنة، واستخدمنا جرد المبيعات اليومية لاختبار Batch Processing، ثم استخدمنا أكثر من نسخة من التطبيق لاختبار Load Distribution.

بعض العمليات تم تنفيذها فعلياً داخل قاعدة البيانات، مثل إنشاء الطلبات وخصم المخزون. أما عمليات أخرى مثل الدفع، إرسال الإيميل، وتوليد الفاتورة فقد تمت محاكاتها، لأن الهدف الأساسي من المشروع هو اختبار السلوك غير الوظيفي وليس ربط النظام بخدمات خارجية حقيقية.

المتطلبات الوظيفية Functional Requirements

رغم أن تركيز المشروع الأساسي هو المتطلبات غير الوظيفية، كان لا بد من وجود وظائف أساسية تمثل نظام تجارة إلكترونية مبسط. هذه الوظائف لم تبني كنظام متجر كامل، بل كقاعدة عملية لاختبار المتطلبات المطلوبة.

1. المنتجات

يحتوي المشروع على نموذج **Product** الذي يمثل المنتج داخل النظام. يتضمن المنتج اسماً وسعراً، ويتم استخدامه في عمليات الشراء وإنشاء الطلبات التجريبية.

2. المخزون

يمثل نموذج **Stock** كمية المنتج المتاحة في المخزون. استخدمنا هذا الجزء بشكل أساسي في اختبار Race Condition، حيث يتم إرسال عدة طلبات شراء متزامنة على نفس المنتج للتحقق من سلامة تحديث المخزون.

3. الطلبات

يمثل نموذج **Order** طلب الشراء داخل النظام. يتم إنشاء الطلب عند نجاح عملية الشراء، كما استخدمت الطلبات لاحقاً في اختبار المعالجة الدفعية من خلال توليد عدد كبير من الطلبات المكتملة.

4. عناصر الطلب

يمثل نموذج **OrderItem** المنتجات الموجودة داخل كل طلب، مع الكمية والسعر. هذا النموذج يربط الطلب بالمنتجات التي تم شراؤها، ويجعل عملية الشراء أوضح داخل قاعدة البيانات.

5. السلة

يحتوي المشروع أيضاً على نماذج **Cart** و **CartItem** كسلة تسوق مبدئية. لم تكن السلة محور الاختبارات، لكنها جزء طبيعي من بنية نظام التجارة الإلكترونية.

6. العمليات المحاكاة

بعض الوظائف لم تُنفذ تخدمات حقيقية، بل تمت محاكاتها لخدمة الاختبارات. عملية الدفع تمت محاكاتها لاختبار إدارة الموارد، بينما تمت محاكاة إرسال الإيميل وتوليد الفاتورة لاختبار المعالجة غير المتزامنة. هذا الاختيار كان لأن الهدف من المشروع ليس بناء نظام دفع أو بريد إلكتروني حقيقي، بل اختبار تأثير هذه العمليات على الأداء وزمن الاستجابة واستهلاك الموارد.

7. جرد المبيعات اليومية

تم تنفيذ وظيفة جرد المبيعات اليومية في متطلب Batch Processing. تقوم هذه الوظيفة بمعالجة الطلبات المكتملة على دفعات، وحساب الإيرادات، وتسجيل نتائج كل تشغيل وكل دفعة.

8. توزيع الطلبات

في متطلب Load Distribution تم تنفيذ Load Balancer بسيط يوزع الطلبات على أكثر من نسخة من التطبيق. هذه الوظيفة تساعد على توضيح كيف يمكن تقليل الضغط على نسخة واحدة من التطبيق من خلال توزيع الحمل.

9. حدود المتطلبات الوظيفية

المشروع لا يحتوي على واجهة مستخدم كاملة أو نظام دفع حقيقي أو إرسال بريد إلكتروني فعلي. تم تنفيذ الوظائف الأساسية فقط بالقدر الذي يسمح باختبار المتطلبات غير الوظيفية المطلوبة بشكل واضح وعملي.

المتطلبات غير الوظيفية Non-Functional Requirements

يركز هذا القسم على المتطلبات غير الوظيفية التي تم تطبيقها في المشروع. تم اختيار كل متطلب بناءً على مشكلة عملية يمكن أن تظهر في أنظمة التجارة الإلكترونية، ثم تم تطبيق حل مناسب واختباره عملياً. الهدف من هذا القسم هو توضيح المشكلة، الحل المطبق، وطريقة التحقق من أن الحل أعطى النتيجة المطلوبة.

1. Race Condition Handling

تظهر مشكلة Race Condition عندما يحاول أكثر من طلب الوصول إلى نفس البيانات وتعديلها في نفس الوقت. في مشروعنا تظهر هذه المشكلة عند محاولة عدة مستخدمين شراء نفس المنتج بالتزامن. إذا قرأت عدة طلبات نفس قيمة المخزون قبل تحديثها، يمكن أن يقبل النظام عدد طلبات أكبر من الكمية المتوفرة فعلياً.

لحل هذه المشكلة كانت هناك عدة حلول ممكنة، مثل استخدام Optimistic Locking، أو Distributed Locks، أو Pessimistic Locking. في هذا المشروع اخترنا استخدام Pessimistic Locking على مستوى قاعدة البيانات، وذلك باستخدام:

```
transaction.atomic()
```

```
select_for_update()
```

تضمن `transaction.atomic()` أن عملية الشراء تتم كوحدة واحدة، بينما يقوم `select_for_update()` بقفل صف المخزون أثناء تنفيذ الطلب،

بحيث لا يستطيع طلب آخر تعديله في نفس اللحظة.

تم تنفيذ حالتين للاختبار.

الحالة الأولى هي النسخة غير الآمنة `/orders/place-order-unsafe/`،

والحالة الثانية هي النسخة الآمنة `/orders/place-order/`.

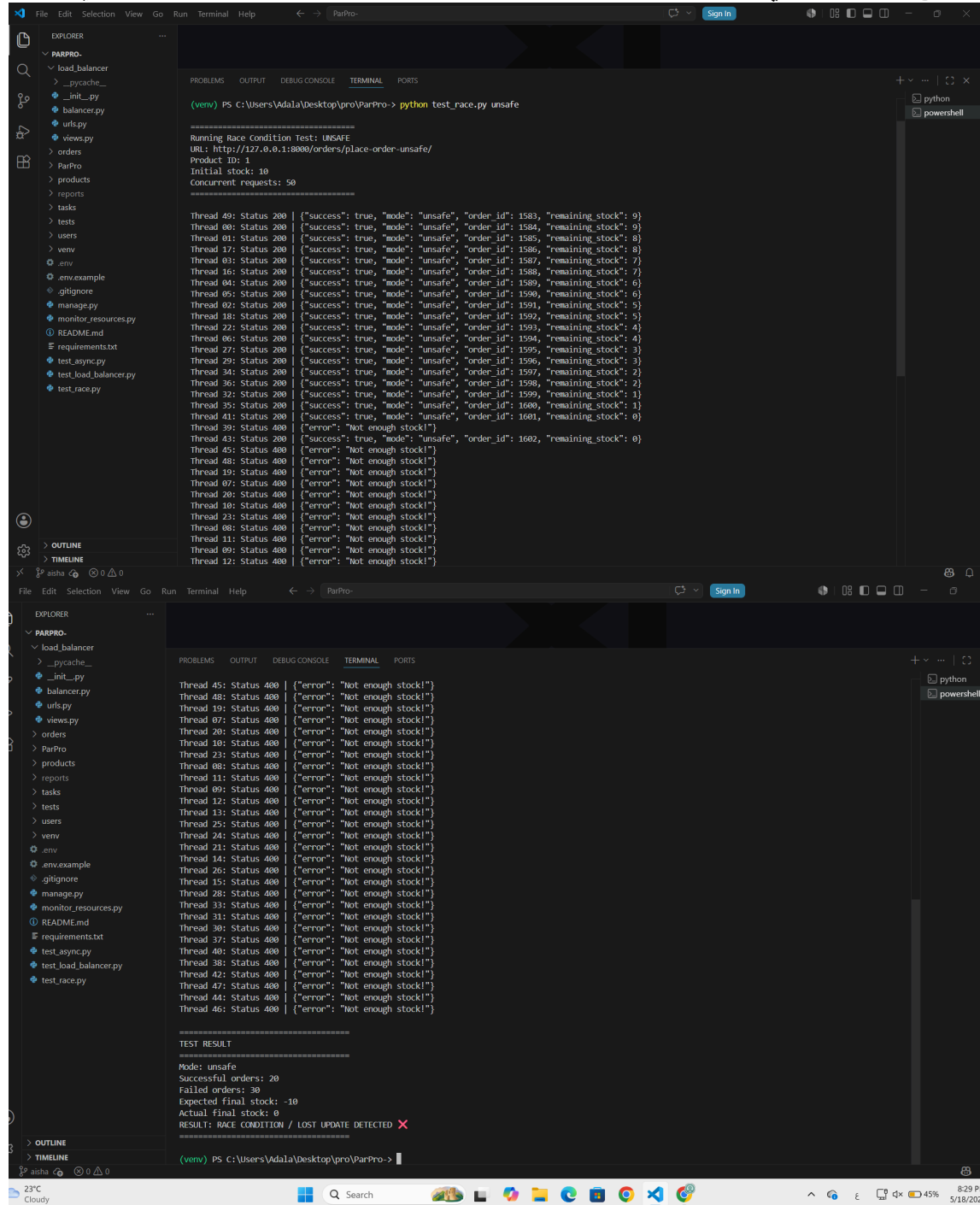
تم اختبار الحالتين باستخدام الملف:

```
test_race.py
```

حيث تم إرسال 50 طلباً متزامناً على منتج يحتوي على مخزون ابتدائي مقداره 10.

Unsafe Race Condition Result

أظهرت نتيجة النسخة غير الآمنة أن عدد الطلبات الناجحة كان أكبر من المخزون المتاح، كما ظهر اختلاف بين المخزون المتوقع والمخزون الفعلي. هذا يدل على حدوث Lost Update بسبب الوصول المتزامن غير المحمي.



```
File Edit Selection View Go Run Terminal Help
ParPro-

EXPLORER
  PARPRO-
    load_balancer
    __pycache__
    __init__.py
    balancer.py
    urls.py
    views.py
    orders
    ParPro
    products
    reports
    tasks
    tests
    users
    venv
    .env
    .env.example
    .gitignore
    manage.py
    monitor_resources.py
    README.md
    requirements.txt
    test_async.py
    test_load_balancer.py
    test_race.py

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python test_race.py unsafe

Running Race Condition Test: UNSAFE
URL: http://127.0.0.1:8000/orders/place-order-unsafe/
Product ID: 1
Initial stock: 10
Concurrent requests: 50

Thread 40: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1583, "remaining_stock": 9}
Thread 00: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1584, "remaining_stock": 9}
Thread 01: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1585, "remaining_stock": 8}
Thread 17: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1586, "remaining_stock": 8}
Thread 03: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1587, "remaining_stock": 7}
Thread 16: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1588, "remaining_stock": 7}
Thread 04: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1589, "remaining_stock": 6}
Thread 05: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1590, "remaining_stock": 6}
Thread 02: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1591, "remaining_stock": 5}
Thread 18: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1592, "remaining_stock": 5}
Thread 22: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1593, "remaining_stock": 4}
Thread 06: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1594, "remaining_stock": 4}
Thread 27: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1595, "remaining_stock": 3}
Thread 29: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1596, "remaining_stock": 3}
Thread 34: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1597, "remaining_stock": 2}
Thread 36: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1598, "remaining_stock": 2}
Thread 32: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1599, "remaining_stock": 1}
Thread 35: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1600, "remaining_stock": 1}
Thread 41: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1601, "remaining_stock": 0}
Thread 39: Status 400 | {"error": "not enough stock!"}
Thread 43: Status 200 | {"success": true, "mode": "unsafe", "order_id": 1602, "remaining_stock": 0}
Thread 45: Status 400 | {"error": "not enough stock!"}
Thread 42: Status 400 | {"error": "not enough stock!"}
Thread 19: Status 400 | {"error": "not enough stock!"}
Thread 07: Status 400 | {"error": "not enough stock!"}
Thread 20: Status 400 | {"error": "not enough stock!"}
Thread 10: Status 400 | {"error": "not enough stock!"}
Thread 23: Status 400 | {"error": "not enough stock!"}
Thread 08: Status 400 | {"error": "not enough stock!"}
Thread 13: Status 400 | {"error": "not enough stock!"}
Thread 09: Status 400 | {"error": "not enough stock!"}
Thread 12: Status 400 | {"error": "not enough stock!"}
Thread 13: Status 400 | {"error": "not enough stock!"}
Thread 25: Status 400 | {"error": "not enough stock!"}
Thread 24: Status 400 | {"error": "not enough stock!"}
Thread 21: Status 400 | {"error": "not enough stock!"}
Thread 14: Status 400 | {"error": "not enough stock!"}
Thread 26: Status 400 | {"error": "not enough stock!"}
Thread 15: Status 400 | {"error": "not enough stock!"}
Thread 28: Status 400 | {"error": "not enough stock!"}
Thread 33: Status 400 | {"error": "not enough stock!"}
Thread 31: Status 400 | {"error": "not enough stock!"}
Thread 30: Status 400 | {"error": "not enough stock!"}
Thread 27: Status 400 | {"error": "not enough stock!"}
Thread 40: Status 400 | {"error": "not enough stock!"}
Thread 38: Status 400 | {"error": "not enough stock!"}
Thread 42: Status 400 | {"error": "not enough stock!"}
Thread 47: Status 400 | {"error": "not enough stock!"}
Thread 44: Status 400 | {"error": "not enough stock!"}
Thread 46: Status 400 | {"error": "not enough stock!"}

TEST RESULT
Mode: unsafe
Successful orders: 20
Failed orders: 30
Expected final stock: -10
Actual final stock: 0
RESULT: RACE CONDITION / LOST UPDATE DETECTED ✖

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
```

Safe Transaction Result

بعد تطبيق القفل باستخدام `select_for_update()`، نجح فقط عدد الطلبات المطابق للمخزون المتاح، وفشلت باقي الطلبات برسالة عدم كفاية المخزون. كما أصبح المخزون الفعلي مطابقا للمخزون المتوقع، مما يثبت أن المشكلة تم حلها.

```
File Edit Selection View Go Run Terminal Help
ParPro

EXPLORER
  PARPRO-
    load_balancer
      __pycache__
        __init__.py
        balancer.py
        urls.py
        views.py
      orders
      ParPro
      products
      reports
      tasks
      tests
      users
      venv
      .env
      .env.example
      .gitignore
      manage.py
      monitor_resources.py
      README.md
      requirements.txt
      test_async.py
      test_load_balancer.py
      test_race.py

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python test_race.py safe

Running Race Condition Test: SAFE
URL: http://127.0.0.1:8000/orders/place-order/
Product ID: 1
Initial stock: 10
Concurrent requests: 50

Thread 49: Status 200 | {"success": true, "mode": "safe", "order_id": 1603}
Thread 01: Status 200 | {"success": true, "mode": "safe", "order_id": 1604}
Thread 46: Status 200 | {"success": true, "mode": "safe", "order_id": 1605}
Thread 48: Status 200 | {"success": true, "mode": "safe", "order_id": 1606}
Thread 44: Status 200 | {"success": true, "mode": "safe", "order_id": 1607}
Thread 40: Status 200 | {"success": true, "mode": "safe", "order_id": 1608}
Thread 00: Status 200 | {"success": true, "mode": "safe", "order_id": 1609}
Thread 43: Status 200 | {"success": true, "mode": "safe", "order_id": 1610}
Thread 12: Status 200 | {"success": true, "mode": "safe", "order_id": 1611}
Thread 06: Status 200 | {"success": true, "mode": "safe", "order_id": 1612}
Thread 18: Status 400 | {"error": "Not enough stock!"}
Thread 15: Status 400 | {"error": "Not enough stock!"}
Thread 16: Status 400 | {"error": "Not enough stock!"}
Thread 13: Status 400 | {"error": "Not enough stock!"}
Thread 17: Status 400 | {"error": "Not enough stock!"}
Thread 05: Status 400 | {"error": "Not enough stock!"}
Thread 22: Status 400 | {"error": "Not enough stock!"}
Thread 19: Status 400 | {"error": "Not enough stock!"}
Thread 23: Status 400 | {"error": "Not enough stock!"}
Thread 20: Status 400 | {"error": "Not enough stock!"}
Thread 11: Status 400 | {"error": "Not enough stock!"}
Thread 08: Status 400 | {"error": "Not enough stock!"}
Thread 03: Status 400 | {"error": "Not enough stock!"}
Thread 09: Status 400 | {"error": "Not enough stock!"}
Thread 34: Status 400 | {"error": "Not enough stock!"}
Thread 29: Status 400 | {"error": "Not enough stock!"}
Thread 28: Status 400 | {"error": "Not enough stock!"}
Thread 30: Status 400 | {"error": "Not enough stock!"}
Thread 31: Status 400 | {"error": "Not enough stock!"}
Thread 27: Status 400 | {"error": "Not enough stock!"}
Thread 32: Status 400 | {"error": "Not enough stock!"}
Thread 33: Status 400 | {"error": "Not enough stock!"}
Thread 35: Status 400 | {"error": "Not enough stock!"}
Thread 36: Status 400 | {"error": "Not enough stock!"}
Thread 37: Status 400 | {"error": "Not enough stock!"}
Thread 38: Status 400 | {"error": "Not enough stock!"}
Thread 39: Status 400 | {"error": "Not enough stock!"}
Thread 41: Status 400 | {"error": "Not enough stock!"}
Thread 42: Status 400 | {"error": "Not enough stock!"}
Thread 45: Status 400 | {"error": "Not enough stock!"}
Thread 47: Status 400 | {"error": "Not enough stock!"}

TEST RESULT
=====
Mode: safe
Successful orders: 10
Failed orders: 40
Expected final stock: 0
Actual final stock: 0
RESULT: CONSISTENT ✓

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
```

Resource Management and Capacity Control.2

في بعض أنظمة التجارة الإلكترونية توجد عمليات ثقيلة مثل معالجة الدفع. إذا تم تنفيذ كل عملية دفع مباشرة بدون أي تحكم بعدد العمليات المتزامنة، فقد يؤدي ذلك إلى استهلاك كبير للذاكرة والمعالج، خصوصاً عند وجود عدد كبير من الطلبات في نفس الوقت.

الحلول الممكنة لهذه المشكلة تشمل إنشاء Thread لكل طلب، أو استخدام Cached Thread Pool، أو استخدام Fixed Thread Pool. اخترنا في المشروع استخدام Fixed Thread Pool لأنه يسمح بتحديد عدد ثابت من العمليات التي تعمل بالتوازي، ويمنع تشغيل عدد غير محدود من المهام الثقيلة.

تم تنفيذ حالتين:

`/orders/process – payment – uncontrolled/`

`/orders/process – payment – controlled/`

في الحالة الأولى يتم تنفيذ عملية الدفع مباشرة مع كل طلب. أما في الحالة الثانية فيتم تمرير عمليات الدفع إلى `ThreadPoolExecutor` بعدد `workers` محدود، وهو 5 `workers` في مشروعنا.

تم اختبار المتطلب باستخدام JMeter لإرسال عدد كبير من الطلبات المتزامنة، بالإضافة إلى ملف `monitor_resources.py` لمراقبة استهلاك CPU و RAM. كما تم استخدام endpoint خاص لقراءة عدد عمليات الدفع المتزامنة:

`/orders/payment-metrics/`

في الحالة غير المضبوطة ظهر أن عدد عمليات الدفع المتزامنة وصل إلى عدد كبير من الطلبات، مثل 50 عملية متزامنة. هذا يوضح أن النظام يسمح بتشغيل جميع العمليات الثقيلة في نفس الوقت.

```
Label: uncontrolled_50
Output CSV: reports\resource_uncontrolled_50_20260518_210248.csv
CPU warning limit: 90.0%
RAM warning limit: 85.0%
Press CTRL + C to stop monitoring.

[2026-05-18 21:02:49] CPU: 1.9% | RAM: 73.1%
[2026-05-18 21:02:50] CPU: 4.8% | RAM: 73.2%
[2026-05-18 21:02:51] CPU: 6.5% | RAM: 73.3%
[2026-05-18 21:02:52] CPU: 17.0% | RAM: 80.4%
[2026-05-18 21:02:53] CPU: 3.3% | RAM: 81.1%
[2026-05-18 21:02:54] CPU: 1.0% | RAM: 81.1%
[2026-05-18 21:02:55] CPU: 4.3% | RAM: 81.1%
[2026-05-18 21:02:56] CPU: 4.1% | RAM: 81.1%
[2026-05-18 21:02:57] CPU: 5.1% | RAM: 80.8%
[2026-05-18 21:02:58] CPU: 5.8% | RAM: 78.6%
[2026-05-18 21:02:59] CPU: 8.0% | RAM: 74.2%
[2026-05-18 21:03:00] CPU: 5.7% | RAM: 73.4%
[2026-05-18 21:03:01] CPU: 2.6% | RAM: 73.3%
[2026-05-18 21:03:02] CPU: 5.2% | RAM: 73.3%
[2026-05-18 21:03:03] CPU: 4.4% | RAM: 73.7%
[2026-05-18 21:03:04] CPU: 3.9% | RAM: 73.9%

Monitoring stopped.
Results saved to: reports\resource_uncontrolled_50_20260518_210248.csv
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro> (Invoke-WebRequest -uri "http://127.0.0.1:8000/orders/payment-metrics/" -Method GET).metrics

uncontrolled_active      : 0
uncontrolled_max_active  : 40
uncontrolled_total_requests : 40
controlled_active        : 0
controlled_max_active    : 0
controlled_total_requests : 0

(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
(venv) PS C:\Users\Adala\Desktop\pro\ParPro>
```


resource_management_test.jmx (C:\Users\Adala\Desktop\pro\ParPro-tests\jmeter\resource_management_test.jmx) - Apache JMeter (5.6.3)

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse...

Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
POST uncon...	50	5833	1	8004	2935.28	20.00%	6.0/sec	6.22	1.10	1069.8
TOTAL	50	5833	1	8004	2935.28	20.00%	6.0/sec	6.22	1.10	1069.8

☐ Include group name in label? ☒ Save Table Header

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->

resource_management_test.jmx (C:\Users\Adala\Desktop\pro\ParPro-tests\jmeter\resource_management_test.jmx) - Apache JMeter (5.6.3)

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename: Browse...

Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughp...	Received ...	Sent KB/s...
POST uncon...	50	5833	7095	7716	7745	8004	1	8004	20.00%	6.0/sec	6.22	1.10
TOTAL	50	5833	7095	7716	7745	8004	1	8004	20.00%	6.0/sec	6.22	1.10

☐ Include group name in label? ☒ Save Table Header

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->

بعد تطبيق Thread Pool، أصبح عدد عمليات الدفع المتزامنة محدوداً بعدد العمال، أي 5 عمليات فقط في نفس الوقت.

```

[2026-05-18 21:06:51] CPU: 2.5% | RAM: 75.0%
[2026-05-18 21:06:52] CPU: 3.3% | RAM: 75.0%
[2026-05-18 21:06:53] CPU: 5.7% | RAM: 75.0%
[2026-05-18 21:06:54] CPU: 3.6% | RAM: 75.0%
[2026-05-18 21:06:55] CPU: 4.4% | RAM: 75.0%
[2026-05-18 21:06:56] CPU: 3.6% | RAM: 75.0%
[2026-05-18 21:06:57] CPU: 3.1% | RAM: 75.0%
[2026-05-18 21:06:58] CPU: 2.0% | RAM: 75.0%
[2026-05-18 21:06:59] CPU: 4.3% | RAM: 75.0%
[2026-05-18 21:07:00] CPU: 7.7% | RAM: 75.0%
[2026-05-18 21:07:01] CPU: 3.3% | RAM: 75.0%
[2026-05-18 21:07:02] CPU: 2.3% | RAM: 75.0%
[2026-05-18 21:07:03] CPU: 2.1% | RAM: 75.0%
[2026-05-18 21:07:04] CPU: 3.2% | RAM: 75.0%
[2026-05-18 21:07:05] CPU: 4.4% | RAM: 75.0%
[2026-05-18 21:07:06] CPU: 3.1% | RAM: 75.0%
[2026-05-18 21:07:07] CPU: 4.0% | RAM: 75.0%
[2026-05-18 21:07:08] CPU: 3.4% | RAM: 74.9%
[2026-05-18 21:07:09] CPU: 5.7% | RAM: 75.0%
[2026-05-18 21:07:10] CPU: 3.8% | RAM: 75.1%
[2026-05-18 21:07:11] CPU: 5.3% | RAM: 75.1%
[2026-05-18 21:07:12] CPU: 2.9% | RAM: 74.1%
[2026-05-18 21:07:13] CPU: 5.0% | RAM: 74.3%
[2026-05-18 21:07:14] CPU: 5.2% | RAM: 74.3%

Monitoring stopped.
Results saved to: reports/resource_controlled_50_20260518_210607.csv
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> (Invoke-RestMethod -Uri "http://127.0.0.1:8000/orders/payment-metrics/" -Method GET).metrics

uncontrolled_active      : 0
uncontrolled_max_active  : 0
uncontrolled_total_requests : 0
controlled_active        : 0
controlled_max_active     : 5
controlled_total_requests : 50

(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
(venv) PS C:\Users\Adala\Desktop\pro\ParPro->
  
```

Summary Report

Name: Summary Report

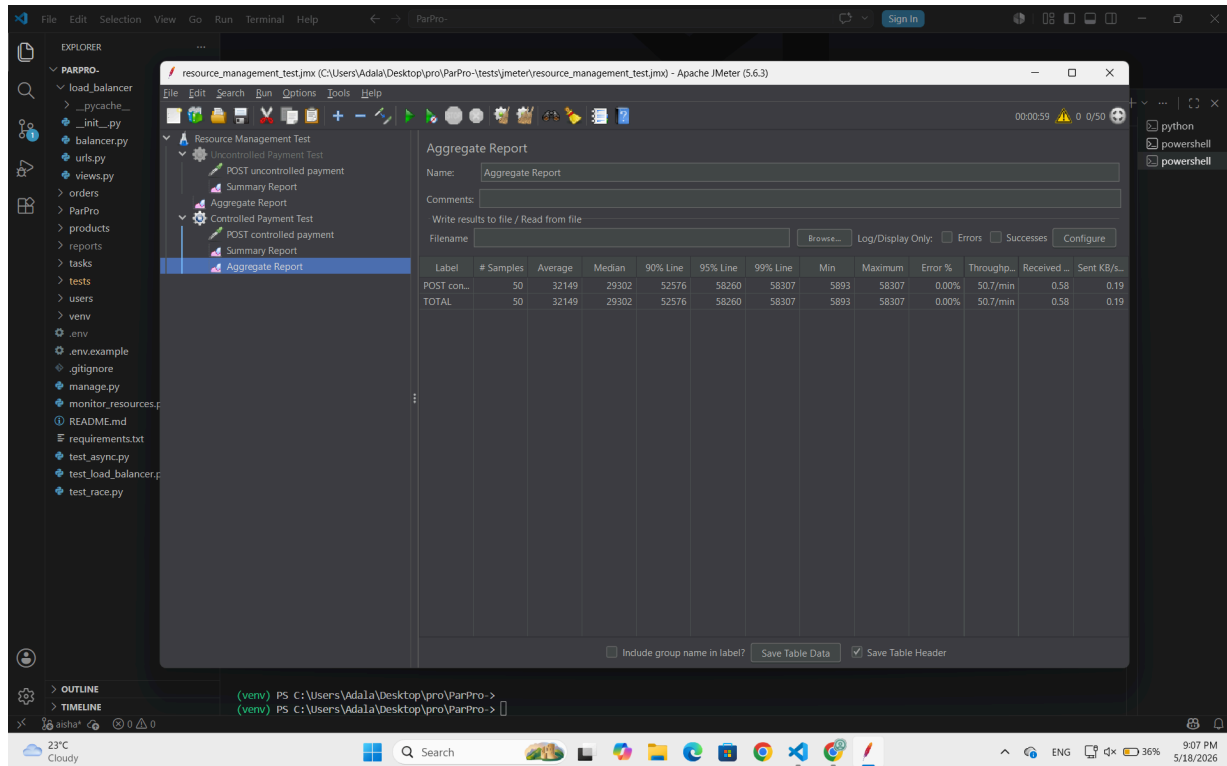
Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
POST contr...	50	32149	5893	58307	16714.09	0.00%	50.7/min	0.58	0.19	708.9
TOTAL	50	32149	5893	58307	16714.09	0.00%	50.7/min	0.58	0.19	708.9

☐ Include group name in label? ☒ Save Table Header



توضح النتائج أن الحل لم يمنع الطلبات من التنفيذ، لكنه نظم طريقة تنفيذ العمليات الثقيلة. في الحالة المضبوطة قد يزيد زمن الاستجابة لبعض الطلبات لأن بعضها ينتظر توفر worker، لكن بالمقابل يبقى استهلاك الموارد تحت السيطرة.

3. Asynchronous Processing

توجد في أنظمة التجارة الإلكترونية مهام ثانوية لا يجب أن تؤثر استجابة المستخدم، مثل إرسال إيميل تأكيد الطلب أو توليد الفاتورة. إذا تم تنفيذ هذه المهام داخل نفس مسار الطلب، فإن المستخدم ينتظر حتى تنتهي جميع المهام قبل الحصول على الرد.

في المشروع تم تنفيذ حالتين:

/orders/place – order – sync/

/orders/place – order – async/

في الحالة المتزامنة **place-order-sync** يتم إنشاء الطلب، ثم ينتظر النظام إرسال الإيميل وتوليد الفاتورة. تمت محاكاة إرسال الإيميل بزمان 2 ثانية، وتوليد الفاتورة بزمان 3 ثوان، لذلك يكون زمن الاستجابة تقريبا 5 ثوان.

في الحالة غير المتزامنة **place-order-async** يتم إنشاء الطلب بسرعة، ثم يتم وضع مهام الإيميل والفاتورة داخل Queue باستخدام **queue.Queue**. بعد ذلك يحصل المستخدم على الاستجابة مباشرة، بينما يقوم Worker في الخلفية بتنفيذ المهام.

تم اختيار **queue.Queue** كحل داخلي مبسط ومناسب لبيئة المشروع. في بيئة إنتاج حقيقية يمكن استبداله بحلول أقوى مثل Redis/Celery أو RabbitMQ.

تم اختبار الحالتين باستخدام الملف:

test_async.py

من خلال الأمر:

python -m tests.test_async compare

أظهرت النتائج أن متوسط زمن الاستجابة في الحالة المتزامنة كان حوالي 5 ثوان، بينما انخفض في الحالة غير المتزامنة إلى أقل من ثانية. هذا يوضح أن نقل المهام الثانوية إلى الخلفية حسن زمن استجابة المستخدم بشكل واضح.

```
File Edit Selection View Go Run Terminal Help
(venv) PS C:\Users\Adala\Desktop\pro\ParPro-> python test_async.py compare

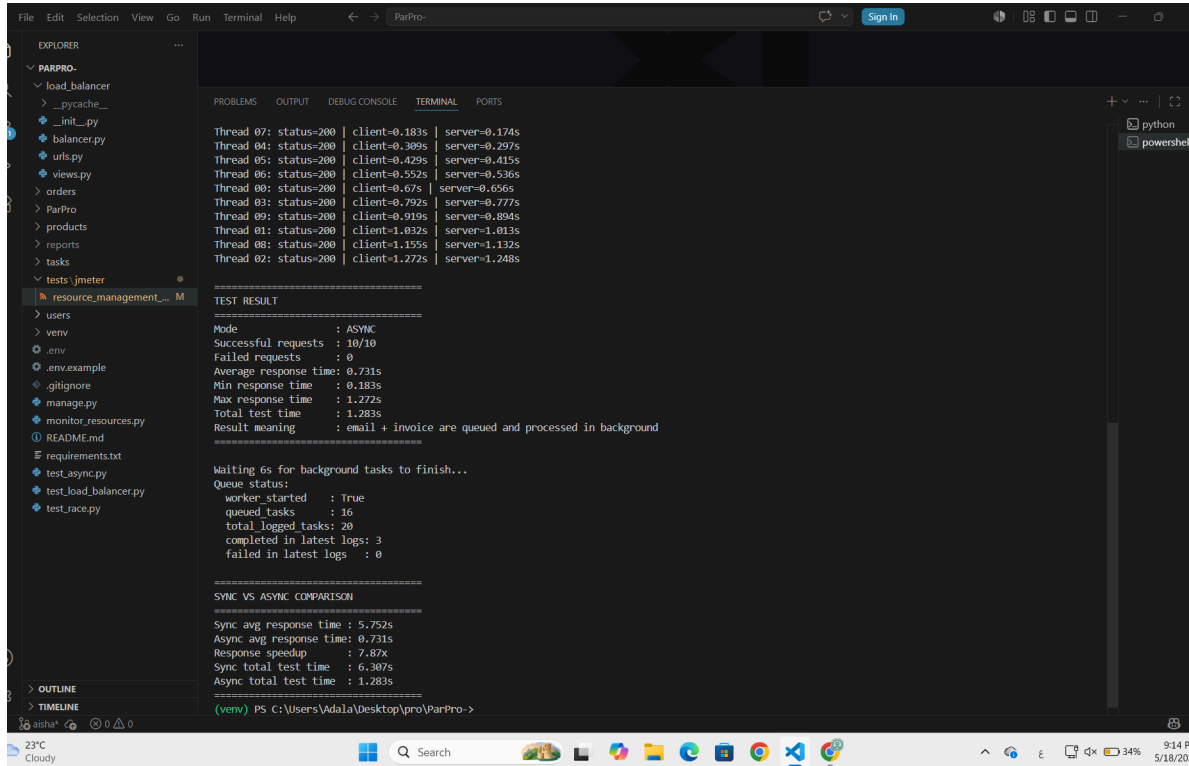
=====
Async Processing Test: SYNC
URL: http://127.0.0.1:8000/orders/place-order-sync/
Concurrent requests: 10
Initial stock: 50
=====

Thread 09: status-200 | client=5.21s | server=5.194s
Thread 01: status-200 | client=5.323s | server=5.303s
Thread 05: status-200 | client=5.448s | server=5.427s
Thread 06: status-200 | client=5.566s | server=5.548s
Thread 04: status-200 | client=5.687s | server=5.666s
Thread 00: status-200 | client=5.808s | server=5.786s
Thread 07: status-200 | client=5.93s | server=5.913s
Thread 02: status-200 | client=6.059s | server=6.03s
Thread 03: status-200 | client=6.182s | server=6.151s
Thread 08: status-200 | client=6.303s | server=6.27s

=====
TEST RESULT
=====
Mode : SYNC
Successful requests : 10/10
Failed requests : 0
Average response time : 5.752s
Min response time : 5.21s
Max response time : 6.303s
Total test time : 6.307s
Result meaning : user waits for email + invoice before response

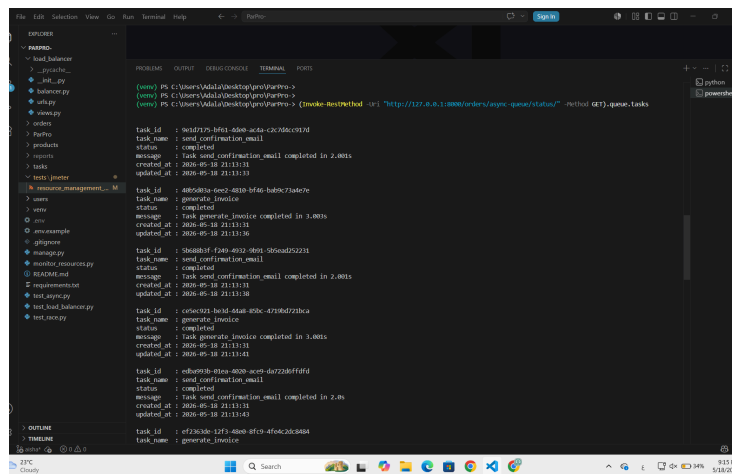
=====
Async Processing Test: ASYNC
URL: http://127.0.0.1:8000/orders/place-order-async/
Concurrent requests: 10
Initial stock: 50
=====

Thread 07: status-200 | client=0.183s | server=0.174s
```



كما تم توفير endpoint لمتابعة حالة الطابور:

/orders/async-queue/status/



توضح حالة الطابور أن مهام مثل `send_confirmation_email` و `generate_invoice` تم تنفيذها في الخلفية بعد إرجاع الاستجابة للمستخدم.

4. Batch Processing

في أنظمة التجارة الإلكترونية قد يحتاج النظام إلى تنفيذ عمليات دورية على عدد كبير من البيانات، مثل جرد المبيعات اليومية. معالجة كل الطلبات دفعة واحدة أو بشكل متسلسل قد لا تكون مناسبة مع زيادة حجم البيانات.

في هذا المشروع تم تطبيق Batch Processing على جرد المبيعات اليومية. تعتمد الفكرة على قراءة الطلبات المكتملة وتقسيمها إلى دفعات Chunks، ثم معالجة كل دفعة بشكل مستقل. تم أيضاً استخدام عدد من workers لمعالجة الدفعات بكفاءة أكبر.

تم تنفيذ عدة أوامر Django لإدارة هذا المتطلب:

```
seed_batch_orders.py
```

```
compare_daily_sales_processing.py
```

```
run_daily_sales_batch.py
```

أولاً تم توليد بيانات اختبار باستخدام:

```
python manage.py seed_batch_orders --count 1000
```

بعد ذلك تم تنفيذ مقارنة بين المعالجة المتسلسلة والمعالجة على دفعات باستخدام:

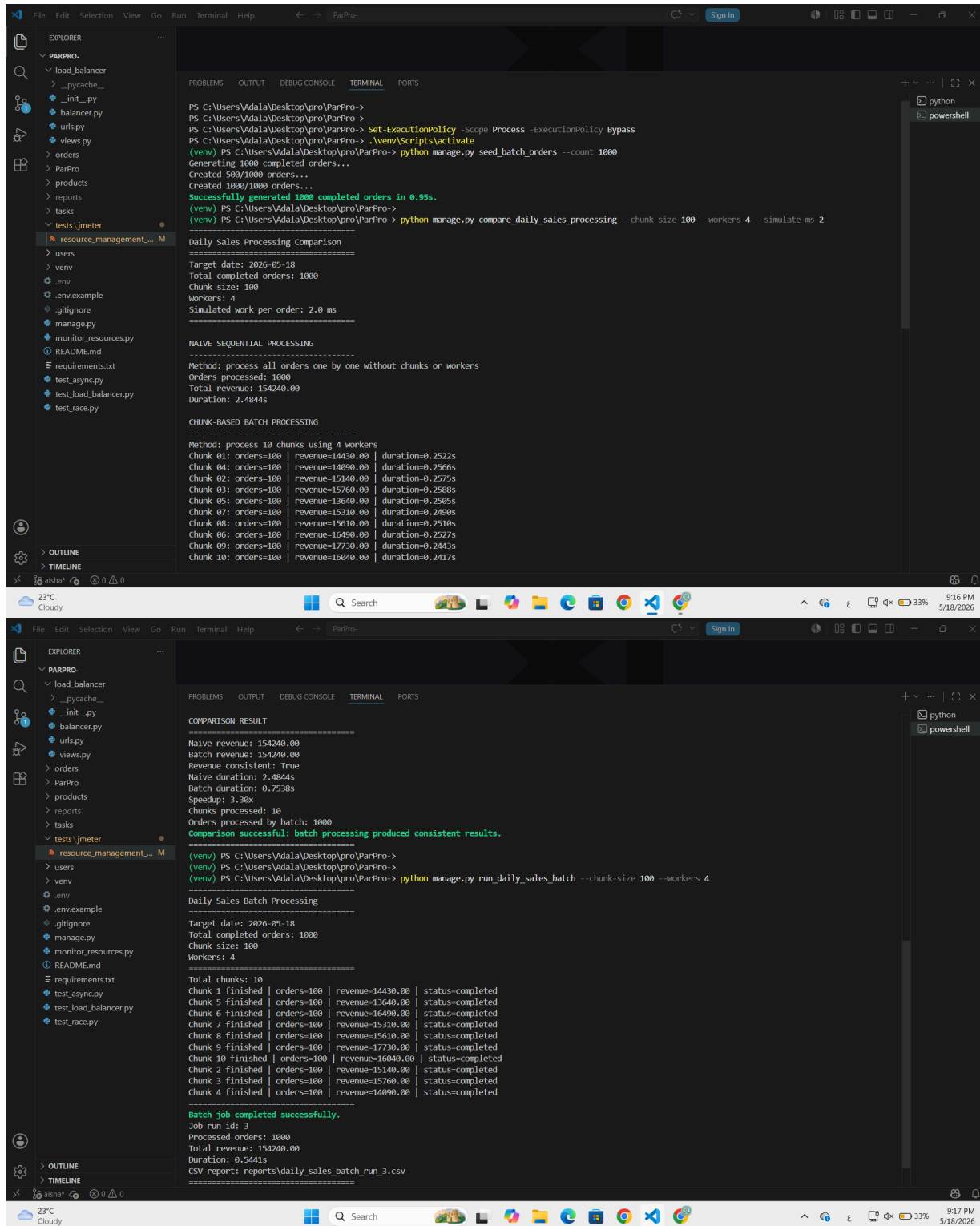
```
python manage.py compare_daily_sales_processing --chunk-size 100 --workers 4  
--simulate-ms 2
```

أظهرت المقارنة أن النتائج الحسابية بقيت متطابقة، مثل إجمالي الإيرادات وعدد الطلبات، لكن المعالجة على دفعات أعطت زمن تنفيذ أفضل عند وجود عمل محاذي لكل طلب. كما ظهرت قيمة Speedup التي توضح التحسن في الأداء.

بعد ذلك تم تشغيل Batch Job الحقيقي:

```
python manage.py run_daily_sales_batch --chunk-size 100 --workers 4
```

أظهر التشغيل أن 1000 طلب تمت معالجتها على 10 دفعات، كل دفعة تحتوي على 100 طلب، مع توليد تقرير CSV داخل مجلد `/reports`. كما تم تخزين نتائج التشغيل والدفعات داخل نماذج `BatchJobRun` و `BatchChunkLog`.



5. Load Distribution

عند زيادة عدد الطلبات على النظام، قد لا يكون تشغيل نسخة واحدة من التطبيق كافياً. لذلك يمكن توزيع الطلبات على أكثر من نسخة من التطبيق باستخدام Load Balancer. الهدف من هذا المتطلب هو إثبات أن الطلبات يمكن أن تتحول إلى أكثر من app instance، ومعرفة مقدار التوزيع بين النسخ.

في المشروع تم تنفيذ Application-Level Load Balancer داخل Django. يعمل ال Load Balancer على المنفذ 8000، ويوزع الطلبات على ثلاث نسخ من التطبيق تعمل على المنافذ التالية:

8001
8002
8003

تم تنفيذ عدة استراتيجيات للتوزيع:

Round Robin
Least Connections
IP Hash

لاختبار المتطلب، تم تشغيل أربع نسخ من Django في أربعة terminals :

```
python manage.py runserver 8000  
python manage.py runserver 8001  
python manage.py runserver 8002  
python manage.py runserver 8003
```

ثم تم تشغيل الاختبار باستخدام:

```
python -m tests.test_load_balancer round_robin
```

أظهرت نتيجة Round Robin توزيعاً متساوياً للطلبات بين الخوادم الثلاثة، حيث حصل كل خادم على 10 طلبات من أصل 30 طلباً. هذا يثبت أن الطلبات تم توزيعها على أكثر من نسخة من التطبيق.


```
(venv) PS C:\Users\Adala\Desktop\pro\ParPro> python test_load_balancer.py round_robin
Reset stats: 200 | {"success": true, "message": "Load balancer stats reset successfully"}

Strategy: ROUND_ROBIN | Requests: 30

Thread 29 + server_1 | status: 200
Thread 13 + server_2 | status: 200
Thread 22 + server_3 | status: 200
Thread 23 + server_2 | status: 200
Thread 06 + server_1 | status: 200
Thread 20 + server_3 | status: 200
Thread 17 + server_2 | status: 200
Thread 14 + server_3 | status: 200
Thread 11 + server_1 | status: 200
Thread 12 + server_2 | status: 200
Thread 10 + server_3 | status: 200
Thread 08 + server_1 | status: 200
Thread 19 + server_3 | status: 200
Thread 04 + server_1 | status: 200
Thread 21 + server_2 | status: 200
Thread 02 + server_1 | status: 200
Thread 00 + server_3 | status: 200
Thread 03 + server_2 | status: 200
Thread 27 + server_2 | status: 200
Thread 25 + server_1 | status: 200
Thread 07 + server_3 | status: 200
Thread 16 + server_1 | status: 200
Thread 01 + server_2 | status: 200
Thread 24 + server_3 | status: 200
Thread 15 + server_2 | status: 200
Thread 18 + server_1 | status: 200
Thread 09 + server_3 | status: 200
Thread 28 + server_1 | status: 200
Thread 26 + server_3 | status: 200
Thread 05 + server_2 | status: 200

=====
DISTRIBUTION RESULT
=====
server_1: ██████████ (10 reqs - 33%)
server_2: ██████████ (10 reqs - 33%)
server_3: ██████████ (10 reqs - 33%)

=====
RESULT: BALANCED
Total handled: 30
=====
```

كما تم اختبار Least Connections:

```
python -m tests.test_load_balancer least_connections
```

أظهرت هذه الاستراتيجية توزيعاً قريباً من التوازن، مع اختلاف بسيط حسب عدد الاتصالات النشطة لحظة إرسال الطلب.

6. Distributed Caching

في أنظمة التجارة الإلكترونية يتم طلب بيانات المنتجات بشكل متكرر، مثل عرض قائمة المنتجات، عرض تفاصيل منتج معين، أو عرض المنتجات الأكثر طلباً. إذا تم تنفيذ كل هذه الطلبات مباشرة من قاعدة البيانات في كل مرة، فقد يؤدي ذلك إلى زيادة عدد قراءات قاعدة البيانات وارتفاع زمن الاستجابة، خصوصاً عند وجود عدد كبير من المنتجات أو عدد كبير من المستخدمين.

لحل هذه المشكلة تم تطبيق Distributed Caching باستخدام Redis. في بيئة التشغيل المحلية تم استخدام Memurai كإدارة متوافق مع Redis. تعتمد الفكرة على تخزين نتائج الاستعلامات المتكررة داخل cache، بحيث تكون أول مرة يتم فيها طلب البيانات Cache Miss ويتم جلبها من قاعدة البيانات، أما الطلبات التالية لنفس البيانات فتكون Cache Hit ويتم إرجاعها من Redis مباشرة دون إعادة قراءة قاعدة البيانات.

تم تنفيذ نسختين لبعض endpoints:

```
/products/list-uncached/  
/products/list-cached/  
/products/<product_id>/detail-uncached/  
/products/<product_id>/detail-cached/  
/products/popular-uncached/  
/products/popular-cached/
```

كما تم توفير endpoints لمتابعة حالة ال cache:

```
/products/cache-metrics/  
/products/cache-metrics/reset/  
/products/cache-clear/
```

تم اختبار المتطلب باستخدام الملف:

```
tests/test_cache.py
```

من خلال الأمر:

```
python -m tests.test_cache
```

أظهرت النتائج أن الطلب الأول على endpoint المخزن كان Cache Miss، بينما أصبح الطلب الثاني Cache Hit. كما ظهر أن زمن الاستجابة في حالة Cache Hit أقل من القراءة المباشرة من قاعدة البيانات، وأن عدد قراءات قاعدة البيانات لا يزداد عند قراءة البيانات من Redis.

```
(base) PS C:\Users\Adala\Desktop\pro\ParPro-> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Adala\Desktop\pro\p
arPro\venv\Scripts\Activate.ps1)
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Adala\Deskt
o\p\pro\ParPro\venv\Scripts\Activate.ps1)
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_cache

Requirement 6 - Distributed Caching Test
=====
==== PRODUCT LIST CACHE TEST ====
-----
1) Uncached product list
-----
Endpoint response time: 0.2589s
Measured client time: 0.2789s
Cache enabled: False
Cache status: N/A
Data source: database
Metrics: {'product_list_hits': 0, 'product_list_misses': 0, 'product_detail_hits': 0, 'product_detail_misses': 0, 'popular_products
_hits': 0, 'popular_products_misses': 0, 'database_reads': 1}
-----
2) Cached product list - first call (MISS)
-----
Endpoint response time: 0.2542s
Measured client time: 0.2626s
Cache enabled: True
Cache status: MISS
Data source: database
Metrics: {'product_list_hits': 0, 'product_list_misses': 1, 'product_detail_hits': 0, 'product_detail_misses': 0, 'popular_products
_hits': 0, 'popular_products_misses': 0, 'database_reads': 2}
-----
3) Cached product list - second call (HIT)
-----
Endpoint response time: 0.0834s
Measured client time: 0.0811s
Cache enabled: True
Cache status: HIT
Data source: redis cache
Metrics: {'product_list_hits': 1, 'product_list_misses': 1, 'product_detail_hits': 0, 'product_detail_misses': 0, 'popular_products
_hits': 0, 'popular_products_misses': 0, 'database_reads': 2}
-----
Product list client-side speedup: 24.99x
=====
==== PRODUCT DETAIL CACHE TEST ====
-----
1) Uncached product detail
-----
Endpoint response time: 0.208s
Measured client time: 0.227s
Cache enabled: False
Cache status: N/A
Data source: database
Metrics: {'product_list_hits': 0, 'product_list_misses': 0, 'product_detail_hits': 0, 'product_detail_misses': 0, 'popular_products
_hits': 0, 'popular_products_misses': 0, 'database_reads': 1}
-----
2) Cached product detail - first call (MISS)
-----
Endpoint response time: 0.2264s
Measured client time: 0.2480s
Cache enabled: True
Cache status: MISS
Data source: database
Metrics: {'product_list_hits': 0, 'product_list_misses': 0, 'product_detail_hits': 0, 'product_detail_misses': 1, 'popular_products
_hits': 0, 'popular_products_misses': 0, 'database_reads': 2}
-----
3) Cached product detail - second call (HIT)
-----
Endpoint response time: 0.0021s
Measured client time: 0.0061s
Cache enabled: True
Cache status: HIT
Data source: redis cache
Metrics: {'product_list_hits': 0, 'product_list_misses': 0, 'product_detail_hits': 1, 'product_detail_misses': 1, 'popular_products
_hits': 0, 'popular_products_misses': 0, 'database_reads': 2}
-----
Product detail client-side speedup: 37.04x
=====
==== POPULAR PRODUCTS CACHE TEST ====
-----
1) Uncached popular products
-----
Endpoint response time: 0.3361s
```

توضح النتيجة أن استخدام Redis Cache ساعد على تقليل الضغط على قاعدة البيانات وتحسين زمن الاستجابة للبيانات التي يتم طلبها بشكل متكرر.

7. Distributed Lock

في بعض الأنظمة توجد مهام ثقيلة يجب ألا تعمل أكثر من مرة في نفس الوقت، مثل توليد تقرير مبيعات يومي أو تنفيذ job خلفي كبير. إذا وصلت عدة طلبات لتشغيل نفس المهمة في نفس اللحظة، فقد يتم تشغيل نفس المهمة عدة مرات بالتوازي، وهذا يؤدي إلى استهلاك موارد غير ضروري وربما إلى نتائج مكررة.

في هذا المتطلب تم تنفيذ Redis Distributed Lock لمنع تشغيل نفس job أكثر من مرة في نفس الوقت. يختلف هذا المتطلب عن المتطلب الأول؛ ففي المتطلب الأول استخدمنا قفلاً على مستوى قاعدة البيانات باستخدام `select_for_update()` لحماية صف المخزون، أما هنا فالمشكلة تتعلق بقفل job كامل على مستوى النظام، لذلك تم استخدام Redis lock.

تم تنفيذ حالتين:

```
/orders/run-sales-report-unsafe/
```

```
/orders/run-sales-report-safe/
```

في الحالة غير الآمنة يمكن لكل الطلبات المتزامنة تشغيل نفس report job في نفس الوقت. أما في الحالة الآمنة، يتم استخدام **SET NX EX** Redis للحصول على lock. إذا حصل طلب واحد على القفل، تبدأ المهمة. أما الطلبات الأخرى فتتوقف مؤقتاً برسالة تفيد بأن القفل موجود.

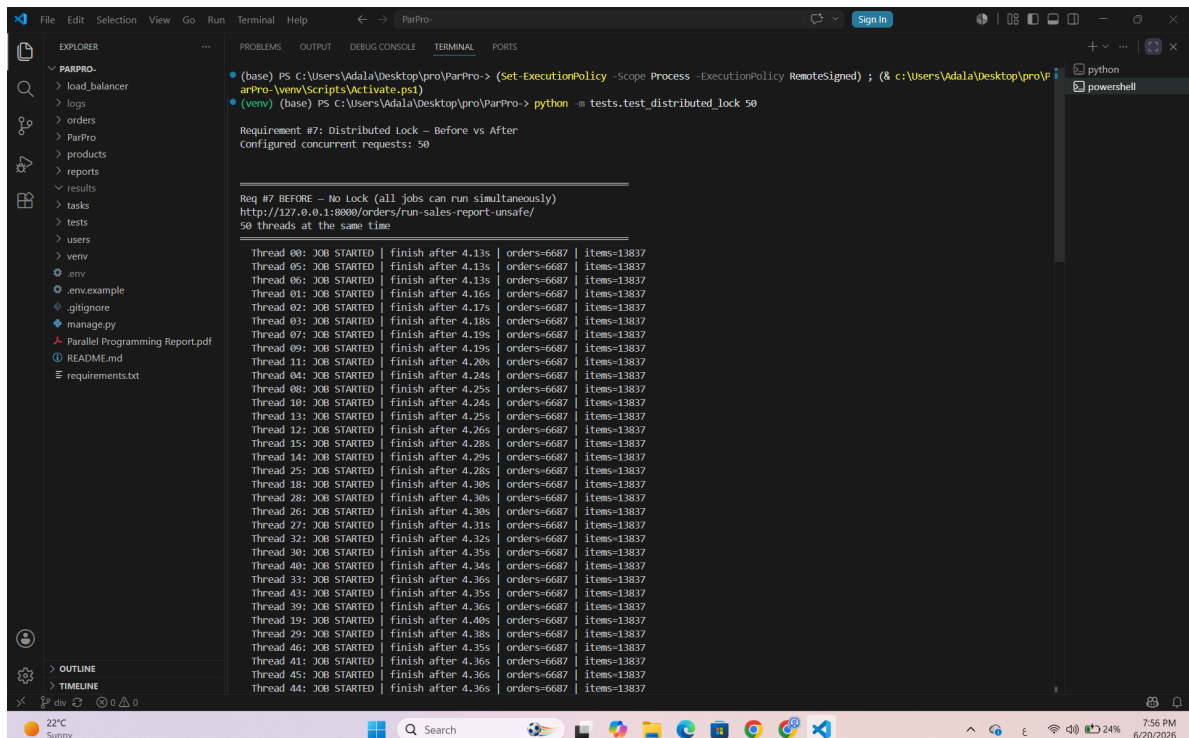
تم اختبار المتطلب باستخدام الملف:

tests/test_distributed_lock.py

من خلال الأمر:

python -m tests.test_distributed_lock 50

أظهرت النتائج أنه قبل استخدام القفل بدأت جميع jobs المتزامنة بالعمل، بينما بعد تطبيق Redis Distributed Lock بدأ job واحد فقط، وتم منع باقي الطلبات من تشغيل نفس المهمة. هذا يثبت أن القفل الموزع نجح في حماية المهمة الثقيلة من التشغيل المتكرر.



```
(base) PS C:\Users\Adala\Desktop\pro\ParPro-> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Adala\Desktop\pro\m\arPro-venv\Scripts\Activate.ps1)
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_distributed_lock 50

Requirement #7: Distributed Lock - Before vs After
Configured concurrent requests: 50

Req #7 BEFORE - No Lock (all jobs can run simultaneously)
http://127.0.0.1:8000/orders/run-sales-report-unsafe/
50 threads at the same time

Thread 00: JOB STARTED | finish after 4.13s | orders=6687 | items=13837
Thread 05: JOB STARTED | finish after 4.13s | orders=6687 | items=13837
Thread 06: JOB STARTED | finish after 4.13s | orders=6687 | items=13837
Thread 01: JOB STARTED | finish after 4.16s | orders=6687 | items=13837
Thread 02: JOB STARTED | finish after 4.17s | orders=6687 | items=13837
Thread 03: JOB STARTED | finish after 4.18s | orders=6687 | items=13837
Thread 07: JOB STARTED | finish after 4.19s | orders=6687 | items=13837
Thread 09: JOB STARTED | finish after 4.19s | orders=6687 | items=13837
Thread 11: JOB STARTED | finish after 4.20s | orders=6687 | items=13837
Thread 04: JOB STARTED | finish after 4.24s | orders=6687 | items=13837
Thread 08: JOB STARTED | finish after 4.25s | orders=6687 | items=13837
Thread 10: JOB STARTED | finish after 4.24s | orders=6687 | items=13837
Thread 13: JOB STARTED | finish after 4.25s | orders=6687 | items=13837
Thread 12: JOB STARTED | finish after 4.26s | orders=6687 | items=13837
Thread 15: JOB STARTED | finish after 4.28s | orders=6687 | items=13837
Thread 14: JOB STARTED | finish after 4.29s | orders=6687 | items=13837
Thread 25: JOB STARTED | finish after 4.28s | orders=6687 | items=13837
Thread 18: JOB STARTED | finish after 4.30s | orders=6687 | items=13837
Thread 28: JOB STARTED | finish after 4.30s | orders=6687 | items=13837
Thread 26: JOB STARTED | finish after 4.30s | orders=6687 | items=13837
Thread 27: JOB STARTED | finish after 4.31s | orders=6687 | items=13837
Thread 32: JOB STARTED | finish after 4.32s | orders=6687 | items=13837
Thread 30: JOB STARTED | finish after 4.35s | orders=6687 | items=13837
Thread 40: JOB STARTED | finish after 4.34s | orders=6687 | items=13837
Thread 33: JOB STARTED | finish after 4.36s | orders=6687 | items=13837
Thread 43: JOB STARTED | finish after 4.35s | orders=6687 | items=13837
Thread 39: JOB STARTED | finish after 4.36s | orders=6687 | items=13837
Thread 19: JOB STARTED | finish after 4.40s | orders=6687 | items=13837
Thread 29: JOB STARTED | finish after 4.38s | orders=6687 | items=13837
Thread 46: JOB STARTED | finish after 4.35s | orders=6687 | items=13837
Thread 41: JOB STARTED | finish after 4.36s | orders=6687 | items=13837
Thread 45: JOB STARTED | finish after 4.36s | orders=6687 | items=13837
Thread 44: JOB STARTED | finish after 4.36s | orders=6687 | items=13837
```

```
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_distributed_lock_50

Thread 22: JOB STARTED | Finish after 4.67s | orders=6687 | items=13837
Thread 24: JOB STARTED | Finish after 4.69s | orders=6687 | items=13837
Thread 23: JOB STARTED | Finish after 4.70s | orders=6687 | items=13837
Thread 21: JOB STARTED | Finish after 4.73s | orders=6687 | items=13837
Thread 31: JOB STARTED | Finish after 4.74s | orders=6687 | items=13837
Thread 34: JOB STARTED | Finish after 4.75s | orders=6687 | items=13837
Thread 35: JOB STARTED | Finish after 4.77s | orders=6687 | items=13837
Thread 36: JOB STARTED | Finish after 4.77s | orders=6687 | items=13837
Thread 37: JOB STARTED | Finish after 4.80s | orders=6687 | items=13837
Thread 38: JOB STARTED | Finish after 4.84s | orders=6687 | items=13837
Thread 42: JOB STARTED | Finish after 4.85s | orders=6687 | items=13837
Thread 47: JOB STARTED | Finish after 4.88s | orders=6687 | items=13837
Thread 48: JOB STARTED | Finish after 4.89s | orders=6687 | items=13837
Thread 49: JOB STARTED | Finish after 4.91s | orders=6687 | items=13837

BEFORE RESULT
-----
Jobs started      : 50
Blocked by Redis Lock : 0
Failed / unexpected : 0

ISSUE DEMONSTRATED: 50 jobs started at the same time.

Req #7 AFTER - Redis Distributed Lock (only one job should run)
http://127.0.0.1:8000/orders/run-sales-report-safe/
50 threads at the same time

Thread 14: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 04: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 06: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 07: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 12: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 18: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 16: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 15: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 11: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 09: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 17: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 13: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 02: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 10: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 09: LOCKED | Redis lock refused request (TTL remaining: 60s)
```

```
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_distributed_lock_50

Thread 25: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 28: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 29: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 31: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 26: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 27: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 30: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 35: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 34: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 37: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 39: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 40: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 33: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 41: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 44: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 45: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 43: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 36: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 49: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 47: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 46: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 32: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 38: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 48: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 42: LOCKED | Redis lock refused request (TTL remaining: 60s)
Thread 00: JOB STARTED | Finish after 4.10s | orders=6687 | items=13837

AFTER RESULT
-----
Jobs started      : 1
Blocked by Redis Lock : 49
Failed / unexpected : 0

FINAL COMPARISON SUMMARY
-----
BEFORE | Concurrent jobs started : 50 | No lock
AFTER  | Concurrent jobs started : 1 | Redis lock
AFTER  | Requests blocked       : 49 | Expected: 49
Req#1 used: select_for_update() <- Database lock
Req#7 used: Redis SET NX EX <- Distributed lock

SOLUTION WORKING: Redis lock allowed exactly one job and blocked the rest.
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro->
```

8. ACID Transaction Integrity

في عملية الشراء داخل نظام التجارة الإلكترونية لا يكفي أن يتم تحديث المخزون فقط، بل يجب أن تتم عدة عمليات معاً كوحدة واحدة. في مشروعنا تتضمن عملية checkout خصم رصيد المستخدم، تحديث المخزون، إنشاء الطلب، وإنشاء عناصر الطلب. إذا فشلت العملية بعد تنفيذ جزء من هذه الخطوات، فقد يبقى النظام في حالة غير متسقة، مثل خصم الرصيد أو المخزون دون إنشاء الطلب.

لحل هذه المشكلة تم تطبيق مبدأ ACID Transaction Integrity باستخدام:

```
transaction.atomic()
```

```
select_for_update()
```

تم إضافة نموذج UserWallet لمحاكاة رصيد المستخدم أثناء عملية الدفع. في الحالة الآمنة يتم قفل محفظة المستخدم وصف المخزون داخل transaction واحدة، بحيث يتم تنفيذ جميع الخطوات معاً أو يتم التراجع عنها كلها عند حدوث خطأ.

تم تنفيذ endpoints خاصة للاختبار:

```
/orders/acid/setup/
```

```
/orders/acid/state/
```

```
/orders/checkout-unsafe/
```

```
/orders/checkout-safe/
```

في الحالة غير الآمنة يمكن أن يحدث فشل بعد خصم الرصيد وتحديث المخزون وقبل إنشاء الطلب، مما يترك النظام في حالة partial update. أما في الحالة الآمنة فإن أي فشل داخل `transaction.atomic()` يؤدي إلى rollback كامل لكل التغييرات.

تم اختبار المتطلب باستخدام الملف:

```
tests/test_acid.py
```

من خلال الأمر:

```
python -m tests.test_acid
```

أظهرت النتائج أن النسخة غير الآمنة يمكن أن تترك تغييرات جزئية، بينما النسخة الآمنة تضمن أن خصم الرصيد، تحديث المخزون، إنشاء الطلب، وإنشاء عناصر الطلب تتم كلها معاً أو يتم إلغاؤها كلها. كما أظهر اختبار التزامن أن النظام بقي متسقاً عند وجود عدة مستخدمين يحاولون الشراء في نفس الوقت.

The screenshot shows a VS Code window with the Explorer sidebar on the left, displaying a project structure for 'PARPRO'. The main editor area shows the 'TERMINAL' output. The terminal contains the following text:

```
(base) PS C:\Users\Adala\Desktop\pro\ParPro-> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Adala\Desktop\pro\p
arPro-venv\Scripts\Activate.ps1)
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_acid

Requirement #8: ACID / Transaction Integrity
Payment + stock update + order creation must commit or rollback together.

=====
SCENARIO 1: UNSAFE FAILURE - partial update problem
=====

Before unsafe checkout failure
-----
Product stock:      10
Single wallet balance: 1000.0
Orders count:      0
OrderItems count:  0
Wallet distribution: [{"balance": 1000.0, 'count': 1}]

Unsafe response:
status: 500
{'success': False, 'mode': 'unsafe', 'transaction used': False, 'error': 'Forced failure after wallet and stock update, before order creation.',
 'inconsistency_risk': True, 'message': 'Failure happened outside transaction. Previous wallet/stock updates may remain saved.'}

=====
After unsafe checkout failure
-----
Product stock:      9
Single wallet balance: 950.0
Orders count:      0
OrderItems count:  0
Wallet distribution: [{"balance": 950.0, 'count': 1}]

Result:
Wallet changed:      True
Stock changed:       True
Order not created:   True
ISSUE DEMONSTRATED: partial update occurred without transaction.

=====
SCENARIO 2: SAFE FAILURE - rollback with transaction.atomic()
=====

Before safe checkout failure
```

The Windows taskbar at the bottom shows the date as 6/20/2026 and the time as 7:59 PM.

The screenshot shows a VS Code window with the Explorer sidebar on the left, displaying a project structure for 'PARPRO'. The main editor area shows the 'TERMINAL' output. The terminal contains the following text:

```
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_acid

Wallet changed:      True
Stock changed:       True
Order not created:   True
ISSUE DEMONSTRATED: partial update occurred without transaction.

=====
SCENARIO 2: SAFE FAILURE - rollback with transaction.atomic()
=====

Before safe checkout failure
-----
Product stock:      10
Single wallet balance: 1000.0
Orders count:      0
OrderItems count:  0
Wallet distribution: [{"balance": 1000.0, 'count': 1}]

Safe failure response:
status: 500
{'success': False, 'mode': 'safe', 'transaction used': True, 'rollback applied': True, 'error': 'forced failure after wallet and stock update, bef
ore order creation.', 'message': 'Failure happened inside transaction.atomic(); wallet, stock, and order changes were rolled back.'}

=====
After safe checkout failure
-----
Product stock:      10
Single wallet balance: 1000.0
Orders count:      0
OrderItems count:  0
Wallet distribution: [{"balance": 1000.0, 'count': 1}]

Result:
Wallet unchanged:    True
Stock unchanged:     True
Order not created:   True
SOLUTION WORKING: rollback restored all changes.

=====
SCENARIO 3: SAFE SUCCESS - all steps commit together
=====

Before safe checkout success
```

The Windows taskbar at the bottom shows the date as 6/20/2026 and the time as 8:00 PM.

```
File Edit Selection View Go Run Terminal Help
ParPro-
Sign In

EXPLORER
PARPRO-
  load_balancer
  logs
  orders
  ParPro
  products
  reports
  results
  tasks
  tests
  users
  venv
  .env
  .env.example
  .gitignore
  manage.py
  Parallel Programming Report.pdf
  README.md
  requirements.txt

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_acid

Result:
Wallet unchanged: True
Stock unchanged: True
Order not created: True
SOLUTION WORKING: rollback restored all changes.

SCENARIO 3: SAFE SUCCESS -- all steps commit together

-----
Before safe checkout success
-----
Product stock: 10
Single wallet balance: 1000.0
Orders count: 0
OrderItems count: 0
Wallet distribution: [{"balance": 1000.0, "count": 1}]

Safe success response:
status: 200
{"success": True, "order_id": 11446, "username": "acid_single_user", "product_id": 3018, "quantity": 1, "total": 50.0, "remaining_wallet_balance": 950.0, "remaining_stock": 9, "mode": "safe", "transaction_used": True, "row_locks_used": ["UserWallet", "Stock"], "message": "Wallet, stock, order, and order item were committed together."}

-----
After safe checkout success
-----
Product stock: 9
Single wallet balance: 950.0
Orders count: 1
OrderItems count: 1
Wallet distribution: [{"balance": 950.0, "count": 1}]

Result:
Wallet deducted: True
Stock deducted: True
Order created: True
OrderItem created: True
SOLUTION WORKING: all checkout steps committed together.

SCENARIO 4: SAFE CONCURRENT CHECKOUT -- 50 users, stock = 20
```

```
File Edit Selection View Go Run Terminal Help
ParPro-
Sign In

EXPLORER
PARPRO-
  load_balancer
  logs
  orders
  ParPro
  products
  reports
  results
  tasks
  tests
  users
  venv
  .env
  .env.example
  .gitignore
  manage.py
  Parallel Programming Report.pdf
  README.md
  requirements.txt

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_acid
SOLUTION WORKING: all checkout steps committed together.

SCENARIO 4: SAFE CONCURRENT CHECKOUT -- 50 users, stock = 20

-----
Before concurrent checkout
-----
Product stock: 20
Single wallet balance: 1000.0
Orders count: 0
OrderItems count: 0
Wallet distribution: [{"balance": 1000.0, "count": 51}]

-----
After concurrent checkout
-----
Product stock: 0
Single wallet balance: 1000.0
Orders count: 20
OrderItems count: 20
Wallet distribution: [{"balance": 950.0, "count": 20}, {"balance": 1000.0, "count": 31}]

Concurrent result:
Total requests: 50
Success count: 20
Failed count: 30
Stock failures: 30
Duration: 0.68s
Expected success: 20
Expected failed: 30
Final stock: 0
Orders created: 20
OrderItems created: 20

Consistency checks:
Success count correct: True
Failure count correct: True
Final stock correct: True
Orders count correct: True
OrderItems count correct: True
SOLUTION WORKING: concurrent checkout stayed consistent.
```

9. Stress Testing

بعد تطبيق عدة حلول لتحسين التزامن والأداء، تم اختبار النظام تحت ضغط 100 مستخدم متزامن. الهدف من هذا المتطلب هو التحقق من قدرة النظام على التعامل مع عدد كبير من الطلبات في نفس الوقت، وليس اختبار endpoint واحد فقط.

تم تصميم الاختبار ليحاكي رحلة مستخدم حقيقية داخل نظام تجارة إلكترونية. يقوم كل مستخدم بتنفيذ العمليات التالية:

cache. عرض قائمة المنتجات المخزنة في.

عرض تفاصيل منتج.

عرض المنتجات الشائعة.

آمن checkout تنفيذ.

تم الاعتماد على بيانات موجودة داخل النظام، مثل users و products و stock، مع إنشاء أو تحديث wallets للمستخدمين عند الحاجة، لأن نموذج UserWallet أضيف لاحقاً ضمن متطلب ACID Transaction.

تم اختبار المتطلب باستخدام الملف:

tests/test_stress.py

من خلال الأمر:

python -m tests.test_stress

```
Requirement #9: Stress Testing
Using existing final seed data: 100 users and 50 products.
Each user performs: list products, product detail, popular products, safe checkout.

Requirement #9: Stress Test Summary

Users: 100
Total requests: 400
Successful requests: 400
Failed requests: 0
Success rate: 100.0%
Checkout success: 380
Checkout failed: 0
Average response time: 390.674 ms
P95 response time: 1668.967 ms
Max response time: 2239.361 ms
Throughput: 166.667 req/s
Status counts: ('200': 400)

Slowest paths:
/products/2834/detail-cached/ | count=2 | avg=1231.985 ms | p95=2239.361 ms | max=2239.361 ms
/products/1383/detail-cached/ | count=2 | avg=1212.455 ms | p95=2087.971 ms | max=2087.971 ms
/products/1460/detail-cached/ | count=2 | avg=1117.983 ms | p95=1879.514 ms | max=1879.514 ms
/products/1463/detail-cached/ | count=2 | avg=1090.216 ms | p95=1842.977 ms | max=1842.977 ms
/products/546/detail-cached/ | count=2 | avg=1005.098 ms | p95=1809.012 ms | max=1809.012 ms
/products/1238/detail-cached/ | count=2 | avg=880.614 ms | p95=1164.447 ms | max=1164.447 ms
/products/list-cached/ | count=100 | avg=798.318 ms | p95=2033.213 ms | max=2105.219 ms
/products/1182/detail-cached/ | count=2 | avg=790.091 ms | p95=1168.707 ms | max=1168.707 ms
/products/2868/detail-cached/ | count=2 | avg=760.985 ms | p95=1242.96 ms | max=1242.96 ms
/products/2177/detail-cached/ | count=2 | avg=743.14 ms | p95=1415.282 ms | max=1415.282 ms
/products/709/detail-cached/ | count=2 | avg=722.659 ms | p95=1170.327 ms | max=1170.327 ms
/products/2434/detail-cached/ | count=2 | avg=641.947 ms | p95=853.935 ms | max=853.935 ms
/products/2444/detail-cached/ | count=2 | avg=540.406 ms | p95=639.562 ms | max=639.562 ms
/products/861/detail-cached/ | count=2 | avg=533.851 ms | p95=625.152 ms | max=625.152 ms
/products/1819/detail-cached/ | count=2 | avg=522.903 ms | p95=617.042 ms | max=617.042 ms
/products/221/detail-cached/ | count=2 | avg=508.372 ms | p95=881.494 ms | max=881.494 ms
/products/823/detail-cached/ | count=2 | avg=378.014 ms | p95=415.088 ms | max=415.088 ms
/products/1250/detail-cached/ | count=2 | avg=364.744 ms | p95=434.158 ms | max=434.158 ms
/products/43/detail-cached/ | count=2 | avg=359.956 ms | p95=429.885 ms | max=429.885 ms
/products/1161/detail-cached/ | count=2 | avg=345.631 ms | p95=418.699 ms | max=418.699 ms
/products/501/detail-cached/ | count=2 | avg=334.386 ms | p95=388.733 ms | max=388.733 ms
/products/879/detail-cached/ | count=2 | avg=334.034 ms | p95=397.479 ms | max=397.479 ms
/products/933/detail-cached/ | count=2 | avg=326.549 ms | p95=373.425 ms | max=373.425 ms
```

```

(base) PS C:\Users\Adala\Desktop\pro\ParPro> python -m tests.test_stress
/Products/1819/detail-cached/ | count=2 | avg=522.903 ms | p95=617.042 ms | max=617.042 ms
/Products/221/detail-cached/ | count=2 | avg=508.372 ms | p95=581.494 ms | max=581.494 ms
/Products/823/detail-cached/ | count=2 | avg=378.014 ms | p95=415.088 ms | max=415.088 ms
/Products/1250/detail-cached/ | count=2 | avg=364.744 ms | p95=434.158 ms | max=434.158 ms
/Products/42/detail-cached/ | count=2 | avg=399.956 ms | p95=429.885 ms | max=429.885 ms
/Products/1161/detail-cached/ | count=2 | avg=345.631 ms | p95=418.699 ms | max=418.699 ms
/Products/501/detail-cached/ | count=2 | avg=334.386 ms | p95=388.733 ms | max=388.733 ms
/Products/879/detail-cached/ | count=2 | avg=334.034 ms | p95=397.479 ms | max=397.479 ms
/Products/933/detail-cached/ | count=2 | avg=326.549 ms | p95=373.425 ms | max=373.425 ms
/Products/2832/detail-cached/ | count=2 | avg=288.717 ms | p95=307.174 ms | max=307.174 ms
/Products/1286/detail-cached/ | count=2 | avg=281.252 ms | p95=286.876 ms | max=286.876 ms
/orders/checkout-safe/ | count=100 | avg=271.187 ms | p95=1113.347 ms | max=1224.909 ms
/Products/2021/detail-cached/ | count=2 | avg=262.916 ms | p95=448.393 ms | max=448.393 ms
/Products/2943/detail-cached/ | count=2 | avg=251.363 ms | p95=468.996 ms | max=468.996 ms
/Products/798/detail-cached/ | count=2 | avg=250.423 ms | p95=456.921 ms | max=456.921 ms
/Products/346/detail-cached/ | count=2 | avg=246.804 ms | p95=443.675 ms | max=443.675 ms
/Products/689/detail-cached/ | count=2 | avg=246.577 ms | p95=451.868 ms | max=451.868 ms
/Products/3001/detail-cached/ | count=2 | avg=239.101 ms | p95=450.445 ms | max=450.445 ms
/Products/692/detail-cached/ | count=2 | avg=224.927 ms | p95=391.909 ms | max=391.909 ms
/Products/2280/detail-cached/ | count=2 | avg=221.681 ms | p95=399.937 ms | max=399.937 ms
/Products/1278/detail-cached/ | count=2 | avg=205.864 ms | p95=346.831 ms | max=346.831 ms
/Products/205/detail-cached/ | count=2 | avg=200.635 ms | p95=329.752 ms | max=329.752 ms
/Products/1289/detail-cached/ | count=2 | avg=197.21 ms | p95=275.985 ms | max=275.985 ms
/Products/1064/detail-cached/ | count=2 | avg=186.329 ms | p95=278.919 ms | max=278.919 ms
/Products/1174/detail-cached/ | count=2 | avg=181.737 ms | p95=304.984 ms | max=304.984 ms
/Products/42/detail-cached/ | count=2 | avg=178.094 ms | p95=329.392 ms | max=329.392 ms
/Products/530/detail-cached/ | count=2 | avg=174.827 ms | p95=288.168 ms | max=288.168 ms
/Products/626/detail-cached/ | count=2 | avg=173.014 ms | p95=277.194 ms | max=277.194 ms
/Products/753/detail-cached/ | count=2 | avg=168.862 ms | p95=293.219 ms | max=293.219 ms
/Products/461/detail-cached/ | count=2 | avg=167.685 ms | p95=289.03 ms | max=289.03 ms
/Products/530/detail-cached/ | count=2 | avg=166.994 ms | p95=281.97 ms | max=281.97 ms
/Products/476/detail-cached/ | count=2 | avg=161.987 ms | p95=262.15 ms | max=262.15 ms
/Products/206/detail-cached/ | count=2 | avg=163.969 ms | p95=291.469 ms | max=291.469 ms
/Products/856/detail-cached/ | count=2 | avg=161.374 ms | p95=256.324 ms | max=256.324 ms
/Products/2290/detail-cached/ | count=2 | avg=154.928 ms | p95=256.179 ms | max=256.179 ms
/Products/765/detail-cached/ | count=2 | avg=152.222 ms | p95=276.622 ms | max=276.622 ms
/Products/229/detail-cached/ | count=2 | avg=150.938 ms | p95=246.651 ms | max=246.651 ms
/Products/1675/detail-cached/ | count=2 | avg=125.258 ms | p95=156.782 ms | max=156.782 ms
/Products/popular-cached/ | count=100 | avg=86.938 ms | p95=203.426 ms | max=1139.73 ms

Saved JSON: results\req9_stress_20260620_200200.json
Saved CSV: results\req9_stress_20260620_200200.csv

RESULT: System handled 100 concurrent users without request failures.
  
```

أظهرت النتائج أن النظام تعامل مع 100 مستخدم مترامز ونفذ حوالي 400 request دون فشل. كما تم قياس متوسط زمن الاستجابة، وقيمة P95، وال throughput، إضافة إلى أخطاء endpoints. هذا الاختبار أعطى مدخلا عمليا للمتطلب التالي، لأن النتائج ساعدت على تحديد الجزء الأبطأ في النظام تحت الضغط.

10. Benchmarking and Bottleneck Analysis.

بعد تنفيذ Stress Testing، تم تحليل النتائج لمعرفة أي جزء من النظام يسبب زمن استجابة أعلى تحت الضغط. أظهرت النتائج أن endpoint قائمة المنتجات الكاملة يمكن أن تكون مكلفة لأنها تعيد كامل catalog المنتجات في response واحد. حتى مع وجود cache، فإن إرجاع عدد كبير من المنتجات يزيد حجم البيانات المرسلة ويزيد زمن serialization والنقل.

لحل هذه المشكلة تم تنفيذ endpoint محسن يعتمد على pagination مع cache:

`/products/list-paginated-cached/?page=1&page_size=100`

الفكرة ليست حذف البيانات أو تقليل حجم قاعدة البيانات، بل تقليل حجم response الواحد. بدلاً من إرجاع كل المنتجات دفعة واحدة، يتم إرجاع أول 100 منتج فقط في الصفحة الأولى، مع إمكانية طلب صفحات أخرى لاحقاً. هذا الأسلوب أقرب إلى ما يحدث في أنظمة التجارة الإلكترونية الحقيقية.

تم استخدام Structured Logging لتسجيل معلومات عن الطلبات، مثل path و status code و duration، داخل ملف log منظم. كما تم استخدام ملف benchmark لمقارنة الأداء قبل وبعد التحسين. تم اختبار المتطلب باستخدام الملف:

tests/test_benchmark.py

من خلال الأمر:

python -m tests.test_benchmark

تمت المقارنة بين:

Before: /products/list-cached/

After: /products/list-paginated-cached/?page=1&page_size=100

أظهرت النتائج أن endpoint المحسن أعطى زمن استجابة أقل، و throughput أعلى، وحجم response أصغر بشكل واضح. في إحدى التجارب انخفض متوسط زمن الاستجابة من حوالي 272 ms إلى حوالي 124 ms، وانخفضت قيمة P95 تقريبا إلى النصف، كما انخفض حجم البيانات المرسلة بحوالي 30 مرة. هذا يثبت أن استخدام pagination مع cache حسن bottleneck المرتبط بإرجاع قائمة المنتجات الكاملة.

```
(venv) (base) PS C:\Users\Adala\Desktop\pro\ParPro-> python -m tests.test_benchmark

Warm-up phases:
Warm-up /products/list-cached/: status=200, time=247.340 ms, bytes=279765
Warm-up /products/list-paginated-cached/?page=1&page_size=100: status=200, time=58.014 ms, bytes=9319

BEFORE: full cached product list
/products/list-cached/
100 requests, max_workers=50

Success: 100/100
Failed: 0
Avg: 256.534 ms
P95: 643.59 ms
Max: 1020.687 ms
Throughput: 94.607 req/s
Avg payload: 279755.91 bytes

AFTER: paginated cached product list
/products/list-paginated-cached/?page=1&page_size=100
100 requests, max_workers=50

Success: 100/100
Failed: 0
Avg: 132.202 ms
P95: 527.333 ms
Max: 538.031 ms
Throughput: 171.233 req/s
Avg payload: 9309.9 bytes

Requirement #10 Final Comparison

Before avg: 256.534 ms
After avg: 132.202 ms
Avg speedup: 1.94x
Before p95: 643.59 ms
After p95: 527.333 ms
P95 speedup: 1.22x
Payload reduction: 30.05x
Saved JSON: resultsreq10_benchmark_20260620_200404.json
Saved CSV: resultsreq10_benchmark_20260620_200404.csv

RESULT: Bottleneck improved after applying cached pagination.
```

```
results > req10_benchmark_20260620_200404.json > {} before
3  "bottleneck": {
4    "problem": "Full product list endpoint returns the whole catalog and becomes slow under stress.",
5    "before_endpoint": "/products/list-cached/",
6    "after_endpoint": "/products/list-paginated-cached/?page=1&page_size=100",
7    "solution": "Use cached pagination to reduce response payload and serialization work."
8  },
9  "before": {
10   "label": "BEFORE: full cached product list",
11   "path": "/products/list-cached/",
12   "requests_count": 100,
13   "success_count": 100,
14   "failed_count": 0,
15   "success_rate_percent": 100.0,
16   "avg_ms": 256.534,
17   "p95_ms": 643.59,
18   "max_ms": 1020.687,
19   "min_ms": 31.992,
20   "elapsed_seconds": 1.057,
21   "throughput_requests_per_sec": 94.607,
22   "avg_payload_bytes": 279755.91,
23   "max_payload_bytes": 279756
24 },
25 "after": {
26   "label": "AFTER: paginated cached product list",
27   "path": "/products/list-paginated-cached/?page=1&page_size=100",
28   "requests_count": 100,
29   "success_count": 100,
30   "failed_count": 0,
31   "success_rate_percent": 100.0,
32   "avg_ms": 132.202,
33   "p95_ms": 527.333,
34   "max_ms": 538.031,
35   "min_ms": 10.883,
36   "elapsed_seconds": 0.584,
37   "throughput_requests_per_sec": 171.233,
38   "avg_payload_bytes": 9309.9,
39   "max_payload_bytes": 9310
40 },
41 "comparison": {
42   "avg_speedup": 1.94,
43   "p95_speedup": 1.22,
44   "payload_reduction_factor": 30.05
```

أدوات الاختبار المستخدمة

تم استخدام عدة أدوات لاختبار المتطلبات غير الوظيفية حسب طبيعة كل مطلب. في بعض المتطلبات تم استخدام سكريبتات Python لمحاكاة الطلبات المتزامنة وقياس زمن الاستجابة، مثل اختبارات Race Condition، المعالجة غير المتزامنة، التخزين المؤقت، القفل الموزع، Stress Testing، ACID Transaction، Benchmarking و.

في متطلب إدارة الموارد تم استخدام JMeter لأنه مناسب لإرسال عدد كبير من الطلبات المتزامنة ومراقبة سلوك النظام تحت الضغط. كما تم استخدام سكريبت monitor_resources.py لمراقبة استهلاك CPU و RAM أثناء تشغيل اختبارات الدفع.

تم استخدام أوامر Django management commands في متطلب Batch Processing، لأنها مناسبة لتنفيذ jobs خلفية ومعالجة بيانات كثيرة على دفعات. كما تم استخدام PostgreSQL كقاعدة بيانات أساسية، واستخدام Memurai لتخادم Redis محلي لدعم التخزين المؤقت والقفل الموزع.

بالإضافة إلى ذلك، تم استخدام Structured Logs وملفات نتائج بصيغة JSON و CSV أثناء اختبارات Stress Testing و Benchmarking. هذه الملفات ساعدت على تحليل زمن الاستجابة، معدل النجاح، أخطاء endpoints، والفرق بين الأداء قبل وبعد التحسين.

الخلاصة

في هذا المشروع تم تطبيق عشرة متطلبات غير وظيفية على نظام تجارة إلكترونية مبسط باستخدام Django و PostgreSQL. تم ربط كل متطلب بسيناريو عملي داخل المشروع، مثل شراء منتج من المخزون، معالجة الدفع، إرسال الإيميل، توليد الفاتورة، جرد المبيعات، توزيع الطلبات على أكثر من خادم، التخزين المؤقت، القفل الموزع، سلامة المعاملات، اختبار الضغط، وتحليل الاختناقات.

أثبتت النتائج أن استخدام المعاملات وقفل الصفوف ساعد على حل مشكلة Race Condition، وأن استخدام Thread Pool ساعد على التحكم بعدد العمليات الثقيلة المتزامنة. كما أظهرت المعالجة غير المتزامنة تحسناً واضحاً في زمن الاستجابة، بينما ساعد Batch Processing على تنظيم معالجة عدد كبير من الطلبات. وأظهر Load Distribution أن الطلبات يمكن توزيعها على أكثر من نسخة من التطبيق بدلاً من الاعتماد على نسخة واحدة فقط.

بالنسبة للمتطلبات الإضافية، أظهر Redis Caching تحسناً في قراءة بيانات المنتجات المتكررة، بينما منع Redis Distributed Lock تشغيل نفس job الثقيل أكثر من مرة في الوقت نفسه. كما حافظت ACID Transactions على اتساق عملية checkout من خلال تنفيذ خصم الرصيد وتحديث المخزون وإنشاء الطلب كوحدة واحدة. وأثبت Stress Testing أن النظام قادر على التعامل مع 100 مستخدم متزامن، بينما أظهر Benchmarking أن استخدام pagination مع cache يقلل حجم البيانات المرسلة ويحسن زمن الاستجابة.

بشكل عام، يوضح المشروع كيف يمكن تحسين جودة النظام ليس فقط من خلال الوظائف التي يقدمها، بل أيضاً من خلال طريقة تعامله مع الضغط، التزامن، الموارد، المهام الخلفية، التخزين المؤقت، وتحليل الأداء. لذلك يمكن اعتبار المشروع نموذجاً مبسطاً لتطبيق مفاهيم مهمة تستخدم في الأنظمة الخلفية الحقيقية.

References

- [1] "Django Documentation, "Database transactions [1]
- [2] "Django Documentation, "QuerySet select_for_update [2]
- [3] "Python Documentation, "queue — A synchronized queue class [3]
- [4] "Python Documentation, "concurrent.futures — ThreadPoolExecutor [4]
- [5] "Apache JMeter Documentation, "User Manual [5]
- [6] "PostgreSQL Documentation, "Explicit Locking [6]